

Problem Statement

Departments in higher education generate vast datasets (curriculum, results, projects, placements, faculty achievements) but these remain fragmented across PDFs, spreadsheets and notices. Students face delays in accessing essential information, while faculty spend time on repetitive queries. Accreditation bodies such as **NBA and NAAC**, and visiting companies during placement drives require structured student data. Currently, extracting such insights is manual, time-consuming and error-prone.

Existing static portals and websites are **non-interactive, non-personalized and lack analytical querying capabilities**. Generic chatbots are unable to process **context-specific academic queries** tied to structured departmental data. Hence, there is a critical need for an **intelligent, retrieval-enabled digital assistant** that can **deliver both informational support and structured insights** for students, faculty, accreditation agencies and recruiters.

Available Datasets:

- **Department:** Curriculum (scheme & syllabus), admission details
- **Student:** Student lists, results, publications, internships, placements, sports, proctored exams, certifications, projects (minor, mini, major), cultural activities, hackathons, industry interactions
- **Faculty:** Faculty list and details, journals, conferences, textbooks, book chapters, timetable, subject allotment, FDP/workshops attended and organized, contributions outside the institution, projects handled, research projects, certifications, achievements

What to Develop?

Multi-Agent RAG System

It will be a chain of specialized agents that break a user query into intent → table selection → column pruning → safe SQL generation → synthesis with retrieved textual evidence.

This reduces search space, token usage, and hallucinations while enabling accurate, auditable answers.

System overview (pipeline)

Front-end / Chat UI receives user query and user persona (student/faculty/recruiter/accreditation).

Intent Agent classifies query into domain(s) (placements, results, curriculum, projects, faculty, accreditation).

Table Agent maps intent → candidate tables (ranked) and proposes them to user (or auto-confirm for low-risk queries).

Column Prune Agent selects minimal relevant columns per chosen table to reduce tokens.

Retriever (hybrid): run fast SQL on selected columns / fetch matching rows from DB + semantic search for textual fields (vector DB).

Query Gen Agent (SQL LLM) produces SQL + an execution plan, using few-shot or a fine-tuned open-source model.

SQL Validator & Safety Agent runs static checks, type checks, permission checks; optionally executes on read-only replica or sandbox and returns results.

Synthesis Agent (RAG LLM) receives DB rows + retrieved docs as evidence, composes final answer with provenance and structured outputs (CSV/JSON/PDF).

Audit & Feedback: log query, evidence, SQL, user feedback for retraining.

System overview (pipeline)

1) Intent Agent

Input: user query + persona.

Output: one or more domain tags with confidence scores.

Method: lightweight classifier (text classification using a small transformer or keyword + model ensemble).

Example tags: placements, results, projects, faculty, syllabus, accreditation.

Use: restrict Table Agent search space drastically.

System overview (pipeline)

2) Table Agent

Input: intent tag(s) + query.

Output: ranked candidate tables with short rationale (e.g., student_result (pass_rate, cgpa) — matches "semester results").

Possible Method:

Maintain a schema registry: for each table keep table_id, description, keywords, sample_rows, sensitivity_level.

Use semantic matching (embed query and table descriptions) + lexical matching on keywords.

Score = $\alpha \cdot \text{semantic_sim}$ + $\beta \cdot \text{keyword_hits}$ + $\gamma \cdot \text{recency/relevance}$.

UX: show top 1–3 suggestions and allow user confirmation. For trusted admin flows, allow auto-select when score > threshold (e.g., 0.9).

System overview (pipeline)

3) Column Prune Agent

Purpose: reduce number of columns/size passed to LLM and SQL generator.

Methods (combined heuristics + model-driven):

Static relevance: map keywords in query to schema column keywords.

Mutual information / feature importance (if analytics exist): prefer high-impact columns.

Greedy pruning with simulation: start with essential columns (ids, timestamps, filter columns), ask a small

LLM prompt “If I could only use 5 columns which are necessary to answer X?” — returns a prioritized list.

Guarantee: always keep at least PK/FKs + any columns referenced in filters/aggregates.

Output: final column set and simple schema snippet.

System overview (pipeline)

3) Column Prune Agent

Purpose: reduce number of columns/size passed to LLM and SQL generator.

Methods (combined heuristics + model-driven):

Static relevance: map keywords in query to schema column keywords.

Mutual information / feature importance (if analytics exist): prefer high-impact columns.

Greedy pruning with simulation: start with essential columns (ids, timestamps, filter columns), ask a small

LLM prompt “If I could only use 5 columns which are necessary to answer X?” — returns a prioritized list.

Guarantee: always keep at least PK/FKs + any columns referenced in filters/aggregates.

Output: final column set and simple schema snippet.

System overview (pipeline)

4) Query Generation Agent (SQL)

Approach: few options depending on risk & scale:

Few-shot prompting to an open model (Llama2/Code-models) or

Fine-tune a code/SQL-capable open model on curated SQL examples and institution-specific patterns (date-handling, academic terms).

Include *Schema + sample rows + column set* in prompt; keep prompt size small by pruning and using schema tokens like:

Table: student(student_id, name, batch, cgpa)

Table: placement(student_id, company, package_lpa, year)

User: Find students batch 2023 with cgpa > 8 and package > 6

Produce: SQL, human explanation, estimated complexity, required permissions.

For Example:

Input: user question + schema

Output: SELECT s.student_id, s.name, s.cgpa, p.company, p.package_lpa FROM student s JOIN placement p ON s.student_id=p.student_id WHERE s.batch=2023 AND s.cgpa>8 AND p.package_lpa>6 ORDER BY p.package_lpa DESC;

System overview (pipeline)

5) SQL Validator & Safety Agent

Static checks:

- Only read operations unless allowed.

- No destructive commands (DELETE, DROP, UPDATE) in generated SQL.

- Table/column ACL checks: does the user have rights?

- Row-level policy enforcement (mask PII if needed).

Runtime checks:

- Run an EXPLAIN or dry-run on a read-only replica to detect heavy scans.

- Add LIMIT fallback for exploratory queries.

If unsafe: return explanation and offer safer reformulation or ask for elevated privileges.

System overview (pipeline)

6) Retriever & Evidence Collector

For textual fields (project abstracts, faculty profiles) do vector search and return top N snippets with highlight + metadata.

For numeric/structured facts, use SQL results.

Always attach provenance: table:row_id or doc:project:123 and timestamps.

7) Synthesis Agent

Input: SQL result rows + textual evidence.

Output: final natural language answer, JSON/table exports, citations, confidence.

Prompt style: evidence-only synthesis with explicit instructions to not hallucinate; include source tags inline.

System overview (pipeline)

Algorithms & heuristics (practical)

Table ranking score:

$\text{score} = 0.6 * \text{cos_sim}(\text{query_embed}, \text{table_embed}) + 0.3 * \text{keyword_overlap_norm} + 0.1 * \text{table_usage_prior}$

Column pruning greedy:

Start with required = {PK/FKs, columns referenced by query filters/aggregates}.

Add columns by descending semantic relevance until token budget reached or coverage threshold met.

Verify by re-running SQL generation prompt; if model refuses due to missing columns, add next column.

Cost-aware planning: Predict execution cost (rows, joins) using row counts & cardinality stats — if too expensive, ask user to filter.

SQL generation best practices and safety

Always restrict to read-only queries by default.

Run generated SQL on a **read-only replica** or use a sandboxed SQL engine with realistic sample data.

Enforce LIMIT and ORDER BY safety for exploratory queries.

Implement deterministic fallback: when generator uncertain, return structured choices (e.g., “Do you mean X or Y?”) instead of guessing.

System overview (pipeline)

Fine-tuning vs Few-shot

Few-shot: quick to start; include domain-specific examples in the prompt for admission dates, batch semantics, academic year ranges, date-handling (semester conversions).

Fine-tune: recommended for production where you:

- Collect 1k–10k paired (NLQ, schema, SQL) examples from real usage + synthetic variations.

- Fine-tune on code/SQL capable models to reduce errors and repetition.

- Continue to supervise with RLHF or human-in-the-loop corrections for high-risk queries.

System overview (pipeline)

Example few-shot prompt for SQL LLM

SYSTEM: You are a SQL generator for the CS Department DB. Use only provided schema. Output valid PostgreSQL.

SCHEMA:

student(student_id PK, name, roll_no, batch, cgpa)

placement(placement_id PK, student_id FK, company, package_lpa, joining_date)

EXAMPLES:

Q: "List top 5 students in batch 2023 by cgpa"

A: "SELECT student_id, name, roll_no, cgpa FROM student WHERE batch=2023 ORDER BY cgpa DESC LIMIT 5;"

Q: "Find students in 2023 with cgpa>8 and package>6"

A: "SELECT s.student_id, s.name, s.cgpa, p.company, p.package_lpa FROM student s JOIN placement p ON s.student_id=p.student_id WHERE s.batch=2023 AND s.cgpa>8 AND p.package_lpa>6 ORDER BY p.package_lpa DESC;"

System overview (pipeline)

Example column-prune prompt (to small LLM)

User query: "Show my semester 4 marks and pending re-evaluations"

Candidate table: student_result(result_id, student_id, course_id, semester, marks, grade, reval_requested, reval_status, exam_date, evaluator_notes)

Return 4–6 columns required to answer.

Expected output: student_id, course_id, semester, marks, grade, reval_status

System overview (pipeline)

Provenance & explainability

Always return the SQL used (or a sanitized version) and top evidence items.

Add a compact audit trail: {"sql":"...", "evidence":[{"type":"table", "ref":"student:123", "score":0.91}], "query_agent_version":"v1.2"}.

Allow "Show me the used rows" button in UI.

System overview (pipeline)

Evaluation metrics & test scenarios

Accuracy of SQL results (compare to ground truth).

Precision@k for table/column selection.

Execution cost (avg query latency, row scanned).

Hallucination rate (answers with false facts).

User satisfaction and follow-up rates.

Create seed tests: 200 domain queries across all personas to measure.

System overview (pipeline)

Tech stack recommendations

Schema registry & OLAP: PostgreSQL / ClickHouse (materialized views).

Vector DB: Qdrant / Milvus / Pinecone.

Search: OpenSearch for notices / PDFs.

LLMs: Start with open models (Llama 2 / Mistral / Code-specialized models) for SQL generation and synthesis; consider fine-tuning.

Orchestration: Python (FastAPI) + async worker queue (Celery/RabbitMQ) for heavy tasks.

Sandbox: read-only replica + query planner (EXPLAIN) + resource limits.

Frontend: React chat UI with evidence pane and approve/deny buttons.

System overview (pipeline)

MVP roadmap (ordered deliverables)

Build schema registry & sample DB with sample rows + sensitivity labels.

Implement Intent Agent (classifier) and a simple Table Agent (semantic matching).

Implement Column Prune Agent with greedy heuristics.

Wire few-shot SQL generator using an open LLM; run on read-only replica.

Add SQL Validator & sandbox execution.

Implement Retriever (vector + SQL) and Synthesis Agent.

Frontend: Chat UI + evidence pane + confirm table selection.

Logging, audit trail, RBAC, and export features.

Collect logs and begin fine-tuning SQL model on real queries.

System overview (pipeline)

Risks & mitigations

Wrong table/column selection → mitigate with user confirmation and visible schema preview.

SQL errors / heavy queries → dry-run, EXPLAIN checks, resource limits.

Hallucinations in synthesis → evidence-only prompts, return “I don’t know” when evidence absent.

System overview (pipeline)

Example interaction (end-to-end)

User: “Give me final-year ML students with internships and packages > 8 LPA.”

Intent Agent → placements, projects, internships.

Table Agent → suggests student, project, placement, internship.

User confirms top suggestion (or auto-accept).

Column Prune → picks student_id, name, batch, project_keywords, internship_org, package_lpa.

Query Gen → produces safe SQL join + LIMIT + ORDER.

Validator runs EXPLAIN → passes.

Execute on replica → returns 12 rows.

Retriever also returns top project abstracts matching “machine learning”.

Synthesis Agent produces CSV + 2-sentence summary + provenance links.

System overview (pipeline)

